MODULE 15 · BLOCK 3 - DATA COLLECTIONS

Strings

Quotes and escapes, indexing and slicing, why strings never change, and the string methods the exam uses.

BLOCK 3 of 4 EXAM WEIGHT 25% MODULE 15 of 28

admin@sthithapragna.com

A string is an **immutable** sequence of characters. Every method that seems to change a string **returns a new one**.

Single or double quotes: the same string

```
1  a = 'single'
2  b = "double"
3  c = "It's fine"            # apostrophe inside double quotes
4  d = 'She said "hi"'        # double quotes inside single quotes
5  e = 'It\'s escaped'        # or escape the quote with a backslash
6  print(c, d, e)
```

3-4 To include one kind of quote, surround the string with the other kind.

5 Or put a backslash in front of the quote, so it no longer ends the string.

Output: It's fine She said "hi" It's escaped

A backslash gives the next character a special meaning

```
1  print("line1\nline2")       # \n is a newline: two lines of output
2  print("a\tb")               # \t is a tab
3  print("back\\slash")        # \\ is one backslash
4  print(len("\n"), len("a\\b"))   # 1 3
```

1-3 The backslash escapes from the normal meaning of the next character.

4 An escape sequence is written with two characters, but it is one character in the string.

A backslash at the very end of a string escapes the closing quote: "back\" is a SyntaxError.

Triple quotes keep the line breaks

```
1  text = """Line one
2  Line two"""
3  print(text)
4  print(len("""a
5  b"""))                  # 3: a, a newline, b
```

1-2 Three quotes, single or double, let a string run over several lines.

4-5 Each line break inside becomes a newline character, and counts towards len.

Indexing and slicing work exactly as for lists

```
1  s = "Python"
2  print(s[0], s[-1], s[1:4])     # P n yth
3  print(s[::-1], s[::2])         # nohtyP Pto
4  print(len(s), "th" in s, "TH" in s)   # 6 True False
5  print(s[6])                    # IndexError: string index out of range
```

2-3 Positive and negative indexes, slices and steps: all the list rules apply.

4 in finds a substring, not just one character. It is case-sensitive.

You cannot change a string. You build a new one

CHANGING A CHARACTER

Not allowed

```
1  s = "cat"
2  s[0] = "b"
```

TypeError: 'str' object does not support item assignment

BUILDING A NEW STRING

The way to do it

```
1  s = "cat"
2  s = "b" + s[1:]
3  print(s)
```

bat. A new string is built, and the name s is moved onto it.

ON THE EXAM

Assigning or deleting a character: TypeError. List methods like append: AttributeError.

Methods return a new string. Capture it

RESULT THROWN AWAY

s is unchanged

```
1  s = "Hello"
2  s.upper()
3  print(s)
```

Hello. upper() built HELLO, but nothing kept it.

RESULT CAPTURED

s now names the new string

```
1  s = "Hello"
2  s = s.upper()
3  print(s)
```

HELLO. The new string is assigned back to s.

THE TRAP

A string method called on a line by itself has no lasting effect.

Five case methods, all returning a new string

METHOD, ON "PYTHON IS FUN"	RESULT
`upper()`	PYTHON IS FUN
`lower()`	python is fun
`capitalize()`	Python is fun: first letter up, all the rest down
`title()`	Python Is Fun: every word starts with a capital
`swapcase()`	PyTHON IS FUN: every letter flipped

THE TRAP

capitalize() lowers everything after the first character, not just the first word.

Break a string into a list, and join a list into a string

```
1  print("a b  c".split())         # ['a', 'b', 'c']   any run of spaces
2  print("a,,b".split(","))       # ['a', '', 'b']    empty piece kept
3  print("-".join(["x", "y", "z"]))   # x-y-z
4  print("".join(["a", "b"]))       # ab
5  print("-".join([1, 2]))          # TypeError: expected str
```

1. With no argument, split cuts on any whitespace, and ignores runs of it.
2. With a separator, every separator cuts, so two in a row leave an empty string.
3-5. join is called on the separator, and every item joined must be a string.

find returns -1. index raises ValueError

FIND

-1 when missing

```
print("banana".find("na"))
print("banana".find("x"))
```

2, then -1. find never raises.

INDEX

ValueError when missing

```
print("banana".index("na"))
print("banana".index("x"))
```

2, then ValueError: substring not found.

THE TRAP

-1 is a valid index, the last character. Test find's result with == -1, never use it as an index.

count, replace, strip, startswith and endswith

```
1  s = "  banana  "
2  print(s.strip(), s.strip().count("a"))    # banana 3
3  print("banana".count("an", 2))           # 1   counting from index 2
4  print("banana".replace("a", "o"))        # bonono
5  print("banana".replace("a", "o", 1))     # bonana   first one only
6  print("xxhixx".strip("x"), "hi".endswith("I"))   # hi False
```

2 strip removes whitespace from both ends only. lstrip and rstrip do one end.

3 A second argument to count, find or index sets where the search starts.

5 A third argument to replace limits how many are replaced.

Tests that return True or False

CALL	RESULT	WHY
`"12".isdigit()`	True	every character is a digit
`"-12".isdigit()` `"1.5".isdigit()`	False	a sign or a point is not a digit
`"".isdigit()`	False	an empty string passes no test
`"abc".isalpha()` `"ab c".isalpha()`	True, then False	a space is not a letter
`"abc123".isalnum()`	True	letters and digits only
`"ABC1".isupper()`	True	only the letters are checked

ON THE EXAM

isdigit() cannot tell you whether int() will succeed: "-12" converts but fails isdigit().

ord() and chr()

```
1  print(ord("a"), ord("A"), ord("0"))     # 97 65 48
2  print(chr(97), chr(ord("p") + 3))       # a s
3  print(ord("c") - ord("a"))              # 2
4  print(chr(ord("g") - 32))               # G
```

2 Code point of p, plus 3, back to a character: p, q, r, s.

4 Each capital's code point is exactly 32 below its lower-case letter.

Which operators work

OPERATOR	EXAMPLE	RESULT
`+ +=`	"ab" + "cd"	abcd: concatenation
`*`	"ab" * 3	ababab: needs an int
`in not in`	"b" in "abc"	True: substring test
`== < >`	"a" < "b"	True: by code point, module 4
`- /`	"abc" - "c"	TypeError: unsupported operand

ON THE EXAM

len(s) gives the length. There is no s.size() or s.length(): those are AttributeErrors.

Is a slice of a string a copy?

- For a **list**, s[:] always builds a new, equal list. That is the model the exam uses.
- For a **string**, CPython skips the copy: str1[:] is str1 gives True, on 3.12 and 3.14.
- A string cannot change, so nothing except **is** can ever tell the difference.
- On the exam, answer with the model: str2 = str1[:] makes a different but equal string.

> ON THE EXAM
>
> The exam tests the idea that slicing copies. The shortcut is a CPython optimisation, not a language rule.

What to carry into the exam

- Strings are **immutable**. A method on a line by itself changes nothing.
- \n is one character. \\ is one backslash. Use the other quote, or escape it.
- split() cuts on whitespace; split(",") keeps empty pieces; join needs strings.
- **find gives -1; index raises ValueError.**
- capitalize lowers the rest. isdigit fails on "-1". chr(ord(c) + n) shifts letters.

What is the output of this program?

```
1  s = "abcdefg"
2  print(s[-3:], s[1:-1:2], s[::-3])
```

A efg bdf gda

B efg bdf gd

C efg ace gda

D efg bdf adg

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  s = "abcdefg"
2  print(s[-3:], s[1:-1:2], s[::-3])
```

[1:-1:2]: indexes 1, 3, 5. [::-3]: indexes 6, 3, 0.

A `efg bdf gda`

Last three; indexes 1, 3, 5; indexes 6, 3, 0.

B `efg bdf gd`

Stops the backward slice at 3. With no stop given, it runs to the start.

C `efg ace gda`

Starts the second slice at 0. Its start is 1.

D `efg bdf adg`

Reverses the result of the third slice. A negative step already reads backwards.

THE SIGNAL

A negative step with no start begins at the last character, and with no stop runs to the first.

What is the output of this program?

```
1  word = "python"
2  word.upper()
3  word = word.replace("p", "j").capitalize()
4  print(word)
```

A PYTHON

B jython

C Jython

D JYTHON

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  word = "python"
2  word.upper()
3  word = word.replace("p", "j").capitalize()
4  print(word)
```

Line 2 is thrown away. "python" -> "jython" -> "Jython".

A PYTHON

Assumes line 2 changed word. Its result was never kept.

B jython

Forgets the capitalize() at the end of the chain.

C Jython

Line 2 is lost; replace then capitalize are chained and kept.

D JYTHON

Applies line 2 as well. It had no lasting effect.

THE SIGNAL

A string method on a line by itself changes nothing. Chained methods apply left to right.

What is the output of this program?

```
1  parts = "2024-09-26".split("-")
2  print("/".join(parts[::-1]), len(parts))
```

A 2024/09/26 3

B 26/09/2024 3

C 26/09/2024 10

D The code raises a TypeError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  parts = "2024-09-26".split("-")
2  print("/".join(parts[::-1]), len(parts))
```

parts is ['2024', '09', '26']. Reversed: ['26', '09', '2024'].

A `2024/09/26 3`
Ignores the [::-1], which reverses the list of parts.

B `26/09/2024 3`
Split into 3 strings, reverse the list, join with /.

C `26/09/2024 10`
Counts characters. parts is a list of 3 pieces.

D The code raises a TypeError
The pieces are strings, so join accepts them.

THE SIGNAL

split gives a list of strings, so join can always put them back together.

What is the output of this program?

```
s = "mississippi"
print(s.find("ss"), s.index("ss", 3),
      s.count("s"), s.find("x"))
```

A 2 5 2 -1

B 2 2 4 -1

C 2 5 4 -1

D The code raises a ValueError

◆ PAUSE AND DECIDE

What is the output of this program?

```
s = "mississippi"
print(s.find("ss"), s.index("ss", 3),
      s.count("s"), s.find("x"))
```

m-i-s-s-i-s-s-i-p-p-i: "ss" starts at 2 and at 5. Four single s characters.

A `2 5 2 -1`

Counts "ss". The count is of a single "s": 4.

B `2 2 4 -1`

Ignores the start of 3, so finds the first "ss" again.

C `2 5 4 -1`

First ss at 2; from 3 the next is at 5; four s; x missing gives -1.

D The code raises a ValueError

find returns -1 for a missing substring. Only index raises.

THE SIGNAL

find and index agree when there is a match. Only a missing substring separates them.

What is the output of this program?

```
1  code = ""
2  for ch in "abc":
3      code += chr(ord(ch) + 1)
4  print(code.upper())
```

A bcd

B ABC

C The code raises a TypeError

D BCD

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  code = ""
2  for ch in "abc":
3      code += chr(ord(ch) + 1)
4  print(code.upper())
```

a -> b, b -> c, c -> d. code is "bcd", printed upper-cased.

A **bcd**
Forgets the upper() in the print.

B **ABC**
Forgets the + 1 shift.

C **The code raises a TypeError**
ord gives an int, + 1 is int arithmetic, and chr turns it back.

D **BCD**
Each letter shifts one on, then upper() capitalises the result.

THE SIGNAL

ord to a number, arithmetic, chr back to a character: the types line up at each step.

> The goal is a string whose text is: It's "odd"

Which two literals produce exactly that text? (Choose two.)

A `'It's "odd"'`

B `"It's \"odd\""`

C `'It\'s "odd"'`

D `"It's "odd""`

◆ PAUSE AND DECIDE

Which two literals produce exactly that text? (Choose two.)

A `'It's "odd"'`

The apostrophe ends the string after It. SyntaxError.

B `"It's \"odd\""`

Double quotes outside; the apostrophe is fine; inner quotes escaped.

C `'It\'s "odd"'`

Single quotes outside; the apostrophe escaped; double quotes are fine.

D `"It's "odd""`

The second double quote ends the string before odd. SyntaxError.

THE SIGNAL

Only the quote character that surrounds the string needs escaping inside it.

What is the output of this program?

```
1  for s in ["12", "-3", "4.5", "x9"]:
2      print(s.isdigit(), end=" ")
```

A. True True True False

B. True False False False

C. True True False False

D. False False False False

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  for s in ["12", "-3", "4.5", "x9"]:
2      print(s.isdigit(), end=" ")
```

isdigit checks every character. -, . and x are not digits.

A `True True True False`
Treats anything number-like as digits. isdigit checks characters.

B `True False False False`
Only "12" is all digits. The -, the . and the x each fail.

C `True True False False`
Accepts the minus sign. It is not a digit.

D `False False False False`
"12" is made entirely of digits.

THE SIGNAL

isdigit asks whether every character is a digit, not whether the text is a number.

What is the output of this program?

```
1  s = "a\tb\\n"
2  print(len(s))
```

A 7

B 6

C 4

D 5

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  s = "a\tb\\n"
2  print(len(s))
```

Characters: a, tab, b, one backslash, n.

A 7

Counts every typed character. Each escape pair is one character.

B 6

Treats \\n as a backslash plus a newline. \\ ends the escape; n is a letter.

C 4

Treats \\n as one newline. The \\ is used up first.

D 5

a, \t, b, \\ and n: five characters.

THE SIGNAL

Read escapes left to right: each backslash takes exactly the next character with it.